# Sesión 7 de prácticas: Recuperación

**Cosas a tener en cuenta:**

- Cuando sabes de antemano cuántas iteraciones (máximas) vas a hacer es *recomendable* utilizar un bucle *for* en vez de un *while.*
- No te olvides de comentar las funciones y poner las tabulaciones de forma correcta.
- Documenta la cabecera de las funciones como has visto en clase.
- No utilices ChatGPT o hagas alguna de las malas prácticas vistas en clase de teoría ya que bajará tu nota (incluso hasta el suspenso).
- El **único** elemento de consulta permitido son los apuntes del curso. No busques en internet, te copies o utilices ChatGPT sin haberlo pensado e intentado antes (varias veces!!).

**Instrucciones de esta sesión**

En esta sesión vamos a terminar los ejercicios de las sesiones anteriores que no dio tiempo en clase. Se evaluarán los ejercicios uno a uno con el profesor y, además, habrá dos ejercicios nuevos sobre vectores.

## Ejercicios de la Sesión 5

1. Escribe un programa en C que, dados dos valores por el usuario pida al usuario calcular una de las siguientes opciones:
    a. Si esos dos números son primos.
    b. Si esos dos números son primos relativos (su MCD es 1).
    c. Si esos dos números son números perfectos (la suma de sus divisores propios es el mismo número).
    d. Si esos dos números son amigables (si la suma de los divisores propios de un número es el otro número y viceversa).

    **Piensa** cómo se puede modularizar y qué funciones te hacen falta. Haz primero los prototipos y ponlos en común con el profesor.

2. Escribe un programa modular que presente un menú para calcular la potencia, factorial y combinatorio de ciertos valores leídos desde el teclado y muestre en pantalla el resultado de la operación. Haz primero la descomposición modular.
    - El *factorial* de un número entero no negativo *n* se define como la multiplicación de todos los números enteros positivos desde 1 hasta *n*. Ejemplo: 4! = 4x3x2x1 = 24.
    - La *combinatoria* de dos números enteros *n* y *m* no negativos siendo *n*>=*m* se define como
        comb(n, m) = n! / m! (n-m)!

    **Reto:** *Modulariza el menú.*

# Ejercicios de la Sesión 6

1. A partir del ejercicio 3 de la semana pasada sobre realizar un programa modular que pregunte al usuario si desea calcular el máximo común divisor o el mínimo común múltiplo a partir de dos números que éste introducía. En este ejercicio se pide hacer lo siguiente:
    - **Modulariza** este programa con una función *imprimeMenu*() que indique qué quiere hacer el usuario.
    - **Modulariza** también la entrada del programa pasando los valores por referencia.

    *Consejo: para calcular MCD/MCM usa la lógica, en lugar de algoritmos como Euclides.*

2. Realiza un programa modular que permite calcular el **área y el perímetro** de un **cuadrado**, un **círculo** o un **triángulo equilátero**. El programa debe presentar un **menú**, solicitar los **datos necesarios** según la figura seleccionada, hacer los cálculos y mostrar los **resultados de los cálculos**.
   *Reto: Utiliza una sola función para cada figura geométrica que devuelva a la vez el perímetro y el área.*

3. Escribe un programa que simule una mini aventura numérica en la que el jugador pueda **explorar distintos lugares** (bosque, castillo y pueblo), **recogiendo un objeto en cada exploración** y soltando el que ya tenía. El programa debe permitir al usuario **ir visitando lugares** mientras que no indique lo contrario. También debe **mostrar su último objeto recogido** y estar completamente modularizado mediante funciones *void* para mostrar la bienvenida, elegir el destino, recoger un objeto y mostrar el inventario. Este programa debe utilizar para esas funciones **paso por referencia para actualizar directamente la variable que representa el lugar que visitar y último objeto del jugador. Debe indicar también cuántos objetos tiene al final del juego y cuántos veces ha explorado cada lugar.**

```
============================
  BIENVENIDO A LA AVENTURA
============================

¿A dónde deseas ir?
1. Bosque
2. Castillo
3. Pueblo
Elige un número: 1
Exploras el bosque... encuentras una espada oxidada.
```

*Tu inventario actual contiene: espada oxidada*

*¿Deseas seguir explorando? (s/n): s*

*¿A dónde deseas ir?*
*1. Bosque*
*2. Castillo*
*3. Pueblo*
*Elige un número: 2*
*Entras al castillo... hallas una corona dorada.*

*Tu inventario actual contiene: corona dorada*

*¿Deseas seguir explorando? (s/n): s*

*¿A dónde deseas ir?*
*1. Bosque*
*2. Castillo*
*3. Pueblo*
*Elige un número: 3*
*Paseas por el pueblo... consigues una bolsa de monedas.*

*Tu inventario actual contiene: bolsa de monedas*

*¿Deseas seguir explorando? (s/n): n*

*=== Fin del juego ===*
*Has recogido un total de 3 objetos.*
*Tu último objeto fue: bolsa de monedas*

*--- Estadísticas de exploración ---*
*Veces que exploraste el bosque: 1*
*Veces que exploraste el castillo: 1*
*Veces que exploraste el pueblo: 1*

4. Realiza un programa que pida números enteros hasta que se introduzca el cero. El programa debe sumar todos los números pares y los números impares por separado y mostrar estas sumas. No mezcles entrada/salida con cálculos. Utiliza funciones para modularizar la lógica.

5. A partir del ejercicio 5 de la semana pasada, el cual consistía en un programa modular para calcular la potencia, factorial y combinatoria, reescribe ese programa usando recursividad.

6. Modulariza el ejercicio del *guessing game* de las sesiones anteriores utilizando **recursividad**. En este juego, el usuario piensa un número y el **programa intenta adivinarlo** realizando sucesivos intentos. Para ello, crea una función llamada *buscar* que se llame a sí misma.

*Consejo: Convierte el bucle iterativo en una estructura recursiva. Llama a la función buscar mientras no se haya adivinado el número. En cada llamada, la función deberá ajustar el intervalo de búsqueda en función de las respuestas del usuario.*

# Ejercicios **nuevos** sobre vectores (Sesión 7)

1. Escribe un programa en C que **pida al usuario números enteros** y los **almacene en un vector**.
   La lectura terminará **cuando el usuario introduzca el valor 0**, que no debe incluirse en el vector.

   El programa deberá:

   - Calcular la **media (promedio)** de los números **pares** e **impares** por separado, creando vectores de números pares e impares en primera instancia.

   - Determinar el **valor máximo y mínimo** del conjunto de números introducidos, junto con su **posición (índice)** dentro del vector.

   - Mostrar todos los resultados por pantalla.

2. **Modulariza** el programa anterior: implementa funciones para las diferentes tareas (por ejemplo, lectura, cálculo de medias, búsqueda de máximo/mínimo, etc.).